

CogniAiOps: Architecture & Governance FAQ

1■■■ How is RBAC/ABAC actually implemented and enforced across diverse enterprise applications?

CogniAiOps learns and maps existing entitlement systems into a unified manifest. Each action_id defines roles, attribute filters, and contextual gates. Before any workflow runs, the AI Orchestrator checks RBAC/ABAC rules dynamically through Policy Adapters. Governance remains distributed but auditable.

2■■■ How does CogniAiOps ensure RBAC/ABAC policies remain accurate over time?

Every user action—UI trigger or API call—is intercepted and logged. Fine-tuning pipelines detect patterns (e.g., unauthorized clicks) and self-heal context rules. SMEs can edit rules manually in the Training Builder UI, ensuring evolving security alignment.

3■■■ How does the system maintain auditability and compliance?

AAM_Logging.yaml defines what and how to log. Each action is time-stamped, PII masked, and retained as per policy. Violations trigger alerts. This creates full lineage-grade audit trails suitable for regulatory review.

4■■■ How does the Agentic AI platform provide productivity and operational insights?

Every workflow event becomes an analytic signal—tracking time-to-resolution, automation adoption, bottlenecks, and entitlement failures. These analytics drive ROI and identify where further automation yields maximum value.

5■■■ How do external applications like JIRA, PTS, and Marketplaces integrate?

External integrations are manifest-driven. Each workflow defines allowed domains and external APIs with unified RBAC and logging policies. Example: risk escalation automatically creates a JIRA ticket while maintaining compliance.

6■■■ How does CogniAiOps handle enable/disable controls at runtime?

Root and workflow-level manifest controls allow instant suspension or fallback to manual mode. Global safety settings like max concurrent flows and emergency disables ensure system resilience and regulatory control.

7■■■ How does CogniAiOps differentiate between variations of the same workflow?

Each parent action has child actions and variations—e.g., expedited vs. standard approvals—with shared governance but separate parameters. Context-driven branching avoids logic duplication and maintains traceability.

8■■■ How are concurrency, scale, and failover managed?

AAM_Root.yaml defines thresholds for flow concurrency, CPU load, and users. When thresholds breach, automation gracefully degrades to manual mode while firing alerts for proactive incident

management.

9■■■ How does the platform integrate observability and continuous learning?

An observability service tracks latency, quality, and cost per workflow. Failed actions feed retraining pipelines or trigger SME review, enabling adaptive governance based on performance data.

■ What insights can management derive from federated orchestration?

Federated orchestration correlates data across domains (risk, audit, fraud). Management can query insights like “show open issues linked to delayed audits” or “find high-value fraud overlaps,” enabling real-time operational intelligence.

Enterprise-Grade Use Cases

Audit Approval: Approvals for standard, expedited, exception, and international flows — *RBAC + ABAC + Temporal Controls*

Risk Escalation: Threshold or manual escalation flows — *Context rules + Auto/Manual Mode*

Fraud Detection: Hybrid AI–analyst flagged workflows — *PII Masking + Entitlement Enforcement*

External Integrations: Multi-domain API orchestration — *Security Manifest Governance*

Productivity Insights: Workflow and ROI metrics — *Observability-driven Intelligence*

Emergency Governance: Manifest-level disable control — *Root Manifest Override*

Executive Summary

CogniAiOps doesn't just automate workflows — it governs them.

Every click, API call, and decision is validated against its RBAC/ABAC fabric, logged for lineage, and observed for efficiency. Over time, these manifests evolve into a self-governing ecosystem — where governance, security, and learning are built into every workflow, not bolted on later.